Qwen3-Coder-Next Tier 3

Part 13 Detailed Implementation Specification

Benchmark Harness

Engineering specification for the subsystem that measures whether the autonomous coding system is actually improving, or merely producing prettier failures.

Document role	**Implementation specification**
Part	13
Subsystem	Benchmark Harness
Dependency posture	Consumes outputs from execution, evaluation, and reporting; feeds gates and trend ana

Scope note: this document is intentionally not an architecture rewrite. It tells an engineer exactly how to build the benchmark layer that makes progress measurable, reproducible, and hard to fake.

1. Purpose

The Benchmark Harness exists to answer one question with evidence instead of enthusiasm: is the system getting better over time? In a multi-agent coding platform, intuition breaks immediately. A feature can pass a few ad hoc tests, look correct in a demo, and still regress on reliability, token cost, or recovery behavior. The harness makes progress visible by collecting the same measures on every run, comparing them against baselines, and publishing a normalized record of what changed.

This part removes a common failure mode in agent projects: every team member becomes convinced the system has improved because one impressive run succeeded. The harness forces repeatable measurement across suites, environments, and build variants. It is the evidence layer that future parts depend on for release gating, trend analysis, and regression detection.

- Part 6 (Testing & Review) depends on consistent run capture so test outcomes can be compared across revisions.
- Part 7 (Evaluation Layer) depends on benchmark outputs to score quality beyond simple pass/fail.
- Part 12 (Multi-Agent Coordination) depends on benchmark telemetry to identify coordination overhead and retry amplification.
- Part 14 (Deployment & Human Approval) depends on thresholded benchmark results before promoting a build.

Key rule: the harness measures the system; it does not silently optimize the system. If a metric is missing, the answer is not to infer it later from vibes and wishful thinking.

2. Scope

Included

- Task suite registration and metadata.
- Run orchestration for repeatable benchmark executions.
- Metric capture for success rate, test pass rate, token usage, execution time, retry count, evaluator score, worktree discard rate, and human escalation rate.
- Baseline snapshotting and comparison logic.
- Immutable run records and artifact references.
- Report generation for humans and machines.
- Threshold evaluation for pass, warn, and block states.

Excluded

- Planning logic for which benchmark to run, which belongs to the orchestrator.
- The detailed implementation of test agents, review agents, or evaluator logic.
- Model inference internals and prompt design.
- Deployment automation beyond publishing harness results to the approval layer.
- Any attempt to change production code based on benchmark outcomes without an explicit upstream control decision.

In scope	Out of scope
Deterministic run capture	Agent reasoning internals

In scope	Out of scope
Metric aggregation and baselines	Business logic of products under test
Trend reporting	Model training pipelines
Quality gates	Manual experiment notebooks

3. System Overview

High-level flow

```
User / CI / Scheduled job
   -> Benchmark Orchestrator
   -> Suite Resolver
   -> Execution Runner
   -> Metric Collector
   -> Baseline Comparator
   -> Report Publisher
   -> Dashboard / Approval Gate
```

A benchmark request begins as a suite identifier plus a revision or configuration target. The orchestrator resolves the suite, initializes a run record, and instructs the execution runner to invoke the system under controlled conditions. During execution, collectors stream structured events into the harness. The harness converts those events into normalized metrics, compares them to baseline snapshots, and produces a final verdict.

The important distinction is that the harness is not a passive log bucket. It enforces a run contract: if a run is missing required telemetry, the result is incomplete, not magically complete. If a suite cannot be reproduced, the system should say so loudly rather than burying the uncertainty in a nice chart.

Conceptual example

```
Suite: auth-system-regression
  tasks: 24
  runs: 3

Run 1 -> success 0.92, pass 0.88, tokens 14.2k, retries 1.3, verdict PASS
Run 2 -> success 0.88, pass 0.84, tokens 15.0k, retries 1.9, verdict WARN
Run 3 -> success 0.80, pass 0.75, tokens 18.4k, retries 2.7, verdict BLOCK

Comparator -> baseline regression detected -> approval gate notified
```

4. Internal Architecture

The Benchmark Harness is composed of a small number of sharply defined services. Each service owns one part of the measurement pipeline so that failures can be isolated and metrics cannot be accidentally conflated.

Component	Responsibility	Primary failure mode
Suite Registry	Stores benchmark suite definitions, versions, and required fixtures.	Suite drift, missing metadata, broken versioning
Execution Orchestrator	Starts runs, assigns targets, and enforces run boundaries.	Duplicate runs, partial runs, untracked retries
Metric Collector	Consumes structured events and converts them to normalized metrics.	Dropped events, inconsistent units, late emission
Baseline Manager	Loads and snapshots comparison targets for each suite.	Wrong baseline, stale baseline, non-reproducible
Comparator / Gate Evaluator	Decides whether current results are better, acceptable, or blocked.	Threshold misconfiguration, hidden regressions

Component	Responsibility	Primary failure mode
Report Publisher	Writes machine-readable and human-readable summaries.	Incomplete reports, mismatched totals

Required abstractions are deliberately narrow: BenchmarkSuite, BenchmarkRun, MetricSample, BaselineSnapshot, ComparisonResult, and Verdict. Every downstream consumer should be able to reason about a run without parsing raw logs. The harness should publish structured artifacts and treat logs as debugging support, not as the source of truth.

Runtime interaction diagram

```
Benchmark Orchestrator
   -> Run Context
      -> Executor
         -> Agents / tools / worktrees
            -> Event Stream
               -> Collector
                  -> Normalized Metrics
                     -> Comparator
                        -> Verdict + Report
```

5. Project Structure

Recommended directory layout

Directory	Purpose
benchmark/	Top-level benchmark harness runtime, orchestrator, and suite management.
benchmark/suites/	Suite definitions, fixtures, metadata, and versioned run contracts.
benchmark/runners/	Execution adapters for local, CI, and scheduled benchmark runs.
benchmark/collectors/	Event collectors, metric normalization, and aggregation logic.
benchmark/baselines/	Baseline snapshots, comparison policies, and threshold definitions.
benchmark/reports/	Report builders for JSON, markdown, and dashboard payloads.
benchmark/storage/	Persistence adapters for run records, artifacts, and immutable history.
benchmark/gates/	Verdict logic and downstream approval-layer interfaces.
config/	Benchmark profile configuration, thresholds, and environment matrices.
schemas/	Versioned contracts for runs, metrics, baselines, and outputs.
tests/	Unit, integration, replay, and regression fixtures for the harness itself.

The benchmark harness should live beside, not inside, the production execution path. That keeps measurement code from becoming a hidden dependency for normal operations and makes it possible to evolve benchmarks independently of the agents they measure.

6. Data Contracts

Input structures

Contract	Required fields	Notes
BenchmarkSuite	suite_id, version, tasks, environment, thresholds, owner	Versioned and immutable once published.
BenchmarkTask	task_id, prompt, expected_outcome, assertions, timeout	Minimal task payload plus verification logic.

Contract	Required fields	Notes
RunRequest	suite_id, target_revision, env_profile, repetition_count, seed	Must capture everything needed for replay.
MetricSample	run_id, task_id, metric_name, metric_value, unit, timestamp	Normalized across all collectors.

Output structures

Contract	Required fields	Notes
BenchmarkRun	run_id, suite_id, start/end time, status, artifacts	Append-only run record.
ComparisonResult	baseline_id, run_id, deltas, regressions, verdict	Computable by machines and readable by humans.
ReportBundle	json_summary, markdown_summary, artifact_links	Published to dashboards and approval gates.

Task state should be explicit and finite: pending, running, collecting, comparing, published, failed, or aborted. Artifacts are immutable. Updates produce new records, not silent rewrites. The contract design is intentionally boring, because boring contracts are what survive real-world drift.

Illustrative run record

```
BenchmarkRun:
  run_id: br-2026-13-00491
  suite_id: auth-system-regression
  status: comparing
  repetition_count: 3
  target_revision: commit-8f13...
  metrics:
    task_success_rate: 0.91
    test_pass_rate: 0.87
    tokens_used: 14200
    execution_time_s: 47.3
    retry_count: 1.4
    evaluator_score: 0.92
    worktrees_discarded: 1
    human_escalations: 0
```

Configuration format

- Environment profile: local, ci, scheduled, or release-candidate.
- Threshold set: warning and blocking values per metric.
- Sampling policy: every run, sampled runs, or suite-specific overrides.
- Retention policy: artifact lifetime and baseline archive lifetime.
- Replay policy: whether a run can be re-executed with identical seeds and fixtures.

7. State Management

The benchmark harness maintains a small number of durable state categories. The state model should be append-mostly and easy to reconstruct from storage. Do not design around mutable in-memory truth; the harness must survive restarts and parallel execution without losing run lineage.

State category	Owner	Persistence model	Consumed by
Suite catalog	Suite registry service	Versioned metadata store	Orchestrator, CI, release tooling
Run history	Execution orchestrator	Append-only event store	Reports, dashboards, audit tools
Baselines	Baseline manager	Snapshot store with versioning	Comparator, approval gate
Verdicts	Gate evaluator	Stored comparison result	Deployment approval, notifications
Artifact indexes	Report publisher	Metadata index + object store	Humans, tests, and machine readers

State transitions should be monotonic wherever possible. Once a run is marked complete, its content is frozen. If additional analysis is required, publish a derived artifact rather than mutating the original run. This avoids the ancient software tradition of explaining a mismatch by editing the evidence after the fact.

Future agents consume harness state through the published run records and comparison artifacts. The planner can learn which suites are expensive; the evaluator can inspect which quality dimensions are regressing; deployment automation can block promotions on negative trend lines. No future component should need to parse logs to answer a release question.

8. Component Specifications

Component	Inputs	Outputs	Failure conditions	Future integration
Suite Registry	Suite definitions, ownership data	Resolved suite manifest	Missing version, invalid task ref	Part 14 approval policies
Execution Orchestrator	Run request, suite manifest	Active run context, child execution jobs	Duplicate run, timeout, partial fan-out	Part 12 coordinated runs
Metric Collector	Event stream, task telemetry	Normalized metric samples	Dropped messages, malformed events	Part 7 live evaluation scoring
Baseline Manager	Current suite version, policy	Baseline snapshot, comparison target	Stale snapshot, wrong suite match	Trend analysis and release gates
Comparator / Gate	Metrics, baseline, thresholds	Verdict with deltas and regressions	Missing baseline, ambiguous thresholds	Deployment approval layer
Report Publisher	Run record, verdict, artifacts	JSON/Markdown summaries, dashboard payloads	Partial write, broken artifact index	Dashboards and audit logs

Each component must publish telemetry for its own failures. The harness is not useful if it quietly collapses into a generic 'something went wrong' state. At minimum, a failure should tell you which stage failed, which inputs were present, and whether the run can be replayed safely.

Detailed interaction rule

- The collector must never infer a metric value if the source event did not exist.
- The comparator must never compare against an implicit baseline.
- The gate evaluator must never convert missing data into a pass.
- The report publisher must preserve the original run metadata even if downstream rendering fails.

9. Implementation Sequence

Step 1 - Define schemas and suite contracts

Establish immutable benchmark contracts first so every later module speaks the same language. Output: versioned suite schema, run schema, metric schema, and baseline schema.

Dependency note: No collector or runner should be written before the contract exists.

Step 2 - Build storage and run registry

Create append-only storage for runs, artifacts, and baseline snapshots. Output: run store, artifact index, and version lookup utilities.

Dependency note: Depends on schemas.

Step 3 - Implement execution orchestration

Add the logic that starts a run, binds it to a suite, and controls repetition and seeds. Output: orchestrator that can start and finalize runs deterministically.

Dependency note: Depends on storage and suite contracts.

Step 4 - Add metric collection and normalization

Translate raw events into canonical metric records. Output: event collector, unit normalizer, and aggregation routines.

Dependency note: Depends on orchestration and telemetry event shapes.

Step 5 - Add baseline comparison and verdict logic

Compare current run data to stored baselines and evaluate thresholds. Output: comparison engine and pass/warn/block verdicts.

Dependency note: Depends on collected metrics.

Step 6 - Add report publishing

Generate machine-readable and human-readable summaries. Output: JSON summary, markdown report, and dashboard payload.

Dependency note: Depends on comparison results.

Step 7 - Add gate integration

Wire verdicts into deployment and approval workflows. Output: explicit block/pass signals and traceable reasons.

Dependency note: Depends on reports and comparisons.

Step 8 - Add replay and regression fixtures

Create deterministic fixture suites and replay harnesses for self-testing the benchmark system.

Dependency note: Depends on full stack behavior.

10. Validation Strategy

Validation must answer three questions: does the harness record the right data, does it compare that data correctly, and can it reproduce the same result tomorrow? The answer must be proven with tests, not inferred from dashboards.

Test layer	What it verifies	Success condition
Unit tests	Metric normalization, threshold logic, schema validation	Pure functions produce stable outputs from fixed in
Integration tests	End-to-end run from suite resolution to verdict	A full synthetic run yields expected artifacts and ver
Replay tests	Deterministic re-execution with frozen seed and fixture	Second execution matches the first within tolerance

Test layer	What it verifies	Success condition
Regression tests	Known bad cases stay blocked	A deliberately broken revision triggers a blocking ve
Load tests	Parallel runs and artifact writes	No lost records under concurrent benchmark press

An especially important validation is drift detection: the harness itself can regress. Collector changes, schema changes, and report changes should all be checked against golden fixtures. If a rename causes a metric to disappear from historical comparison, the test suite must fail before the dashboard lies about stability.

Validation checklist

- [ ] Baseline snapshot generation is reproducible.
- [ ] Metric units are normalized consistently across suites.
- [ ] Run IDs are unique and stable under retries.
- [ ] Concurrent benchmark runs do not overwrite each other's artifacts.
- [ ] A missing telemetry field fails the run, not the report.

11. Deliverables

Artifact	Description
Suite registry module	Versioned source of benchmark suite definitions and metadata.
Run store	Append-only record of benchmark executions and outcomes.
Metric schema pack	Canonical event and metric definitions for collectors and reports.
Baseline manager	Snapshotting and retrieval for comparison targets.
Comparator engine	Diff and verdict logic against thresholds and baselines.
Reporting package	JSON, markdown, and gate payload generation.
Validation fixtures	Deterministic benchmark data for replay and regression tests.
Config templates	Environment-specific profiles and threshold definitions.

The deliverables must be deployable as independent modules, not a pile of scripts glued together by hope. If an artifact cannot be imported, validated, and replayed, it is not part of the benchmark harness; it is a liability with a filename.

12. Acceptance Criteria

Criterion	How it is verified
Every benchmark run produces a complete immutable record.	Integration test checks store contents after a full run.
Each metric is reported in canonical units.	Fixture suite validates unit normalization and aggregation.
A baseline comparison is reproducible.	Replay test produces the same verdict from the same frozen inputs.
Blocking regressions are detected and surfaced.	Deliberately degraded revision produces a block verdict and report.
Parallel runs remain isolated.	Concurrency test confirms no artifact collisions or mixed run IDs.
Report artifacts are machine-readable and human-readable.	Schema validation plus markdown rendering check.

Criterion	How it is verified
The harness can publish to the approval layer without manual intervention.	End-to-end gate integration test.

A run is not 'complete' merely because it emitted a final line. It is complete only when the run record, artifacts, metrics, comparison result, and verdict all exist and pass schema validation.

13. Common Failure Modes

- Treating logs as the source of truth and metric records as an afterthought.
- Comparing runs against a moving baseline without version control.
- Letting collectors silently drop malformed events.
- Mixing measurements from different environment profiles in one report.
- Hard-coding thresholds that cannot be overridden per suite or release channel.
- Generating pretty dashboards without preserving raw comparison data.
- Allowing retries to inflate success rates without marking the run as retried.

The largest design trap is measurement pollution: metrics from one suite or environment leaking into another. The second largest trap is optimism: deciding that a missing sample probably means success because the chart looked good last week. The harness should be rude to ambiguity. Ambiguity belongs in debugging, not in release decisions.

14. Future Integration

The benchmark harness becomes a shared evidence layer for several later parts. Part 14 consumes verdicts and trend data to decide whether a deployment can proceed. Part 12 uses benchmark outcomes to quantify coordination overhead, retry behavior, and cross-agent inefficiency. Part 11 can use benchmark history to decide which context compression strategies preserve quality. Part 7 can use benchmark metadata to judge whether a solution is merely functional or actually improving the system.

- Part 14: deployment and human approval gating.
- Part 12: multi-agent coordination health measurements.
- Part 11: context compression quality checks.
- Part 10: repository intelligence coverage analysis.
- Part 7: evaluator scoring and wrong-problem detection support.

The integration contract should be one-way: the harness publishes facts; future components consume those facts. The harness must not be overloaded with decision logic that belongs to orchestration or deployment.

15. Completion Checklist

- [ ] Suite definitions are versioned and validated.
- [ ] Run records are immutable and replayable.
- [ ] All required metrics are captured and normalized.
- [ ] Baselines are snapshot-based and comparable by version.
- [ ] Verdicts are generated from explicit thresholds.
- [ ] Reports are written in machine-readable and human-readable formats.

- [ ] Gate integration receives a clear pass/warn/block signal.
- [ ] Fixtures cover success, warning, regression, and partial failure cases.
- [ ] Concurrency safety is tested and proven.
- [ ] No metric depends on unstructured log parsing as the primary source.

Completion means that a release manager, an engineer, or an automated gate can answer the same question from the same artifact set and get the same answer. If the answer changes depending on who asks, the benchmark harness is not finished. It is just decorative software.